

Project: LoanGazeAI

Intelligent Loan Approval Prediction Through ML Gaze

Business Problem: Develop ML models that can help the company for determining whether an applicant is eligible for a loan or not.

1) Business problem understanding:

Dream Housing Finance company deals in all kinds of home loans. They have presence across all urban, semi-urban, and rural areas. Customer first applies for home loan and after that company validates the customer's eligibility for loan.

Company wants to automate the loan eligibility process (real time) based on customer details provided while filling online application form.

These details are: Gender, Marital status, Education, Number of dependents, Income, Loan amount, Credit History and others. To automate this process, they have provided a dataset to identify the customer segments that are eligible for loan amount so that they can specifically target these customers.

```
In [4]: # import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

```
In [5]: # Load Data
data = pd.read_csv('E:\\AI-ML_SB\\5.Classification\\Capstone Project- Classification\\data.csv')
data.head()
```

```
Out[5]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome
0	LP001002	Male	No	0	Graduate	No	5849
1	LP001003	Male	Yes	1	Graduate	No	4583
2	LP001005	Male	Yes	0	Graduate	Yes	3000
3	LP001006	Male	Yes	0	Not Graduate	No	2583
4	LP001008	Male	No	0	Graduate	No	6000

2) Data Understanding

1- Data Exploration

In [8]: `data.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 614 entries, 0 to 613
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID               614 non-null   object
1   Gender                601 non-null   object
2   Married               611 non-null   object
3   Dependents            599 non-null   object
4   Education              614 non-null   object
5   Self_Employed         582 non-null   object
6   ApplicantIncome       614 non-null   int64
7   CoapplicantIncome     614 non-null   float64
8   LoanAmount            592 non-null   float64
9   Loan_Amount_Term      600 non-null   float64
10  Credit_History        564 non-null   float64
11  Property_Area         614 non-null   object
12  Loan_Status           614 non-null   object
dtypes: float64(4), int64(1), object(8)
memory usage: 62.5+ KB
```

In [9]: `data.columns`

Out[9]: Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education', 'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount', 'Loan_Amount_Term', 'Credit_History', 'Property_Area', 'Loan_Status'], dtype='object')

In [10]: `data['Loan_ID'].nunique()`

Out[10]: 614

In [11]: *# Dropped unimportant column as per feature selection*

```
if 'Loan_ID' in data.columns:
    data.drop(columns=['Loan_ID'], inplace=True)
```

In [12]: `data.head()`

Out[12]:

	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	Coapplic
--	--------	---------	------------	-----------	---------------	-----------------	----------

0	Male	No	0	Graduate	No	5849	
1	Male	Yes	1	Graduate	No	4583	
2	Male	Yes	0	Graduate	Yes	3000	
3	Male	Yes	0	Not Graduate	No	2583	
4	Male	No	0	Graduate	No	6000	

< ————— >

In [13]: `data['Gender'].unique()`

```
Out[13]: array(['Male', 'Female', nan], dtype=object)
```

```
In [14]: data['Gender'].value_counts()
```

```
Out[14]: Gender
Male      489
Female    112
Name: count, dtype: int64
```

```
In [15]: data['Married'].unique()
```

```
Out[15]: array(['No', 'Yes', nan], dtype=object)
```

```
In [16]: data['Married'].value_counts()
```

```
Out[16]: Married
Yes      398
No       213
Name: count, dtype: int64
```

```
In [17]: data['Dependents'].unique()
```

```
Out[17]: array(['0', '1', '2', '3+', nan], dtype=object)
```

```
In [18]: data['Dependents'].value_counts()
```

```
Out[18]: Dependents
0       345
1       102
2       101
3+       51
Name: count, dtype: int64
```

```
In [19]: data['Education'].unique()
```

```
Out[19]: array(['Graduate', 'Not Graduate'], dtype=object)
```

```
In [20]: data['Education'].value_counts()
```

```
Out[20]: Education
Graduate      480
Not Graduate   134
Name: count, dtype: int64
```

```
In [21]: data['Self_Employed'].unique()
```

```
Out[21]: array(['No', 'Yes', nan], dtype=object)
```

```
In [22]: data['Self_Employed'].value_counts()
```

```
Out[22]: Self_Employed
No       500
Yes       82
Name: count, dtype: int64
```

Create a new column as per requirement --> Income

```
In [24]: data['Income'] = data['ApplicantIncome'] + data['CoapplicantIncome']
```

```
In [25]: data.drop(columns=['ApplicantIncome', 'CoapplicantIncome'], inplace=True)
```

```
In [26]: data['Income'].describe()      # Continuous var
```

```
Out[26]: count      614.000000
mean      7024.705081
std       6458.663872
min       1442.000000
25%       4166.000000
50%       5416.500000
75%       7521.750000
max       81000.000000
Name: Income, dtype: float64
```

```
In [27]: data['LoanAmount'].describe()
```

```
Out[27]: count      592.000000
mean      146.412162
std       85.587325
min        9.000000
25%      100.000000
50%      128.000000
75%      168.000000
max       700.000000
Name: LoanAmount, dtype: float64
```

```
In [28]: data['Loan_Amount_Term'].unique()
```

```
Out[28]: array([360., 120., 240., nan, 180., 60., 300., 480., 36., 84., 12.])
```

```
In [29]: data['Loan_Amount_Term'].value_counts()
```

```
Out[29]: Loan_Amount_Term
360.0      512
180.0       44
480.0       15
300.0       13
240.0        4
84.0         4
120.0        3
60.0         2
36.0         2
12.0         1
Name: count, dtype: int64
```

```
In [30]: data['Credit_History'].unique()
```

```
Out[30]: array([ 1.,  0., nan])
```

```
In [31]: data['Credit_History'].value_counts()
```

```
Out[31]: Credit_History
1.0      475
0.0       89
Name: count, dtype: int64
```

```
In [32]: data['Property_Area'].unique()
```

```
Out[32]: array(['Urban', 'Rural', 'Semiurban'], dtype=object)
```

```
In [33]: data['Property_Area'].value_counts()
```

```
Out[33]: Property_Area
Semiurban    233
Urban        202
Rural        179
Name: count, dtype: int64
```

```
In [34]: data['Loan_Status'].unique()
```

```
Out[34]: array(['Y', 'N'], dtype=object)
```

```
In [35]: data['Loan_Status'].value_counts()
```

```
Out[35]: Loan_Status
Y        422
N        192
Name: count, dtype: int64
```

```
In [36]: continous = ['Income', 'LoanAmount']
descrete_categorical = ['Gender', 'Married', 'Education', 'Self_Employed', 'Cred
descrete_count = ['Dependents', 'Loan_Amount_Term' ]
```

EDA

For Continous Variable apply describe()

```
In [39]: data[continous].describe()
```

```
Out[39]:
```

	Income	LoanAmount
count	614.000000	592.000000
mean	7024.705081	146.412162
std	6458.663872	85.587325
min	1442.000000	9.000000
25%	4166.000000	100.000000
50%	5416.500000	128.000000
75%	7521.750000	168.000000
max	81000.000000	700.000000

For Continous Variable apply Histogram

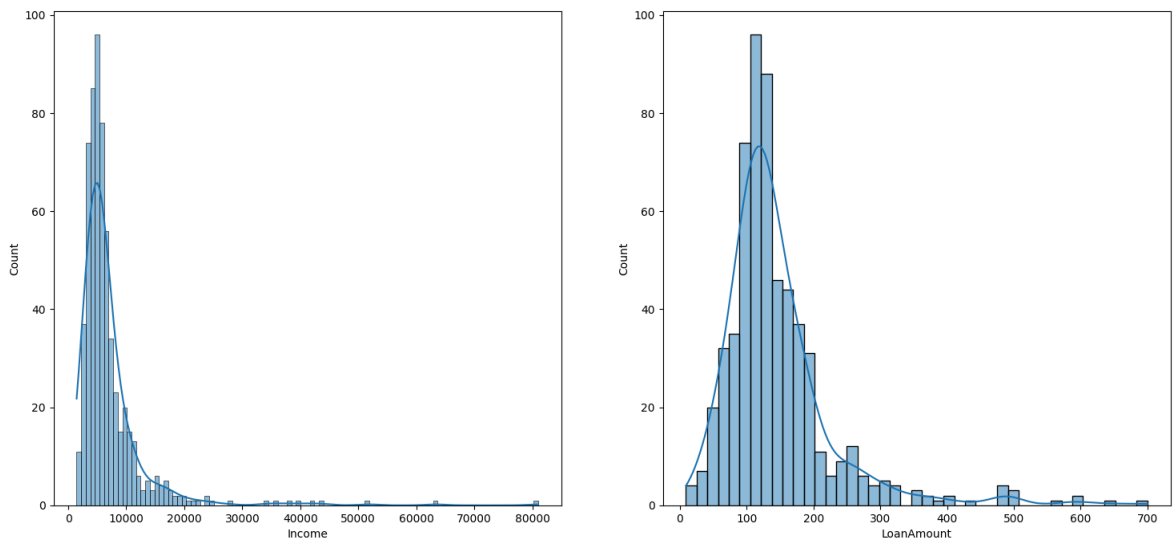
```
In [41]: plt.rcParams['figure.figsize']=(18,8)

plt.subplot(1,2,1)
sns.histplot(data['Income'], kde=True)
```

```
plt.subplot(1,2,2)
sns.histplot(data['LoanAmount'], kde=True)

plt.suptitle('Univariate Analysis on Numerical columns ')
plt.show()
```

Univariate Analysis on Numerical columns

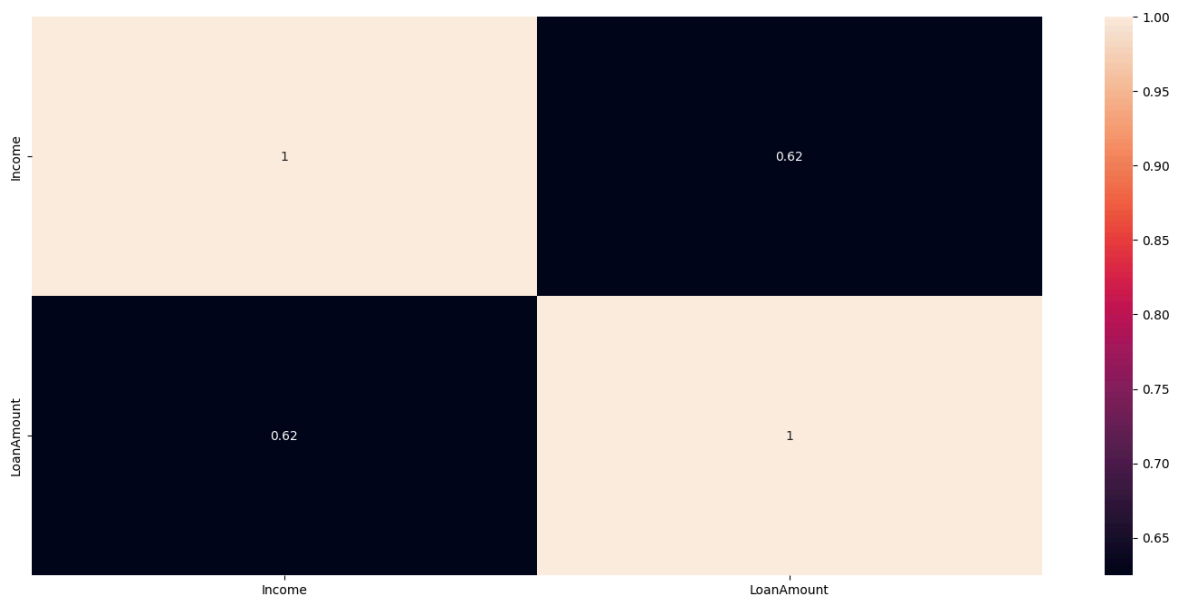


Both Income and LoanAmount are right skewed

--> very less no. of people having high salaries & very less are applying for High loan amount

For Continous Variable applied Correlation matrix (heatmap)

```
In [44]: sns.heatmap(data[continous].corr(), annot=True)
plt.show()
```

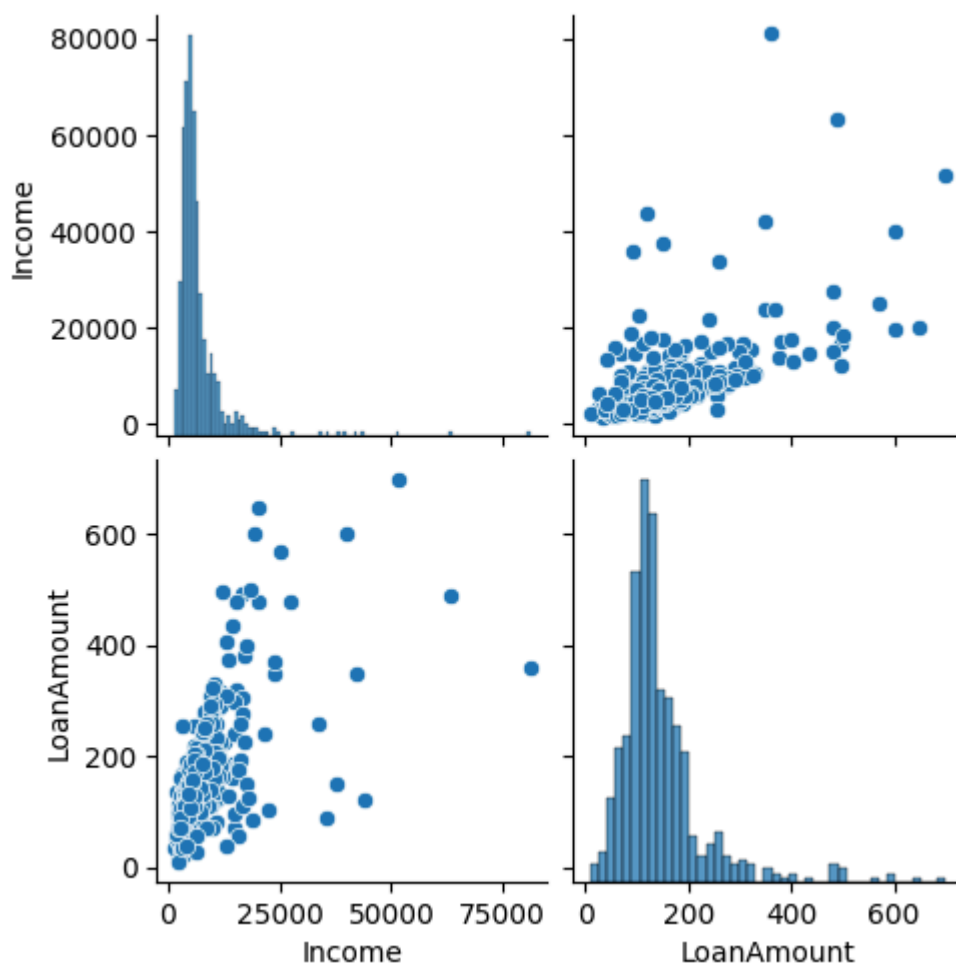


Income & LoanAmount are correlated with 62%

--> Higher the income, Higher the LoanAmount

For Continous Variable applied pairplot to observe scatter plot

```
In [47]: sns.pairplot(data[continuous])
plt.show()
```



Income & LoanAmount bothe are +ve correlated

--> as Income increases, LoanAmount also increases

For Discrete variable apply describe()

```
In [50]: data[descrete_categorical].describe(include='object')
```

```
Out[50]:
```

	Gender	Married	Education	Self_Employed	Property_Area	Loan_Status
count	601	611	614	582	614	614
unique	2	2	2	2	3	2
top	Male	Yes	Graduate	No	Semiurban	Y
freq	489	398	480	500	233	422

For Discrete variable apply count plot

```
In [52]: plt.rcParams['figure.figsize']=(18,8)
plt.subplot(2,3,1)
```

```

sns.countplot(data['Gender'])

plt.subplot(2,3,2)
sns.countplot(data['Married'])

plt.subplot(2,3,3)
sns.countplot(data['Education'])

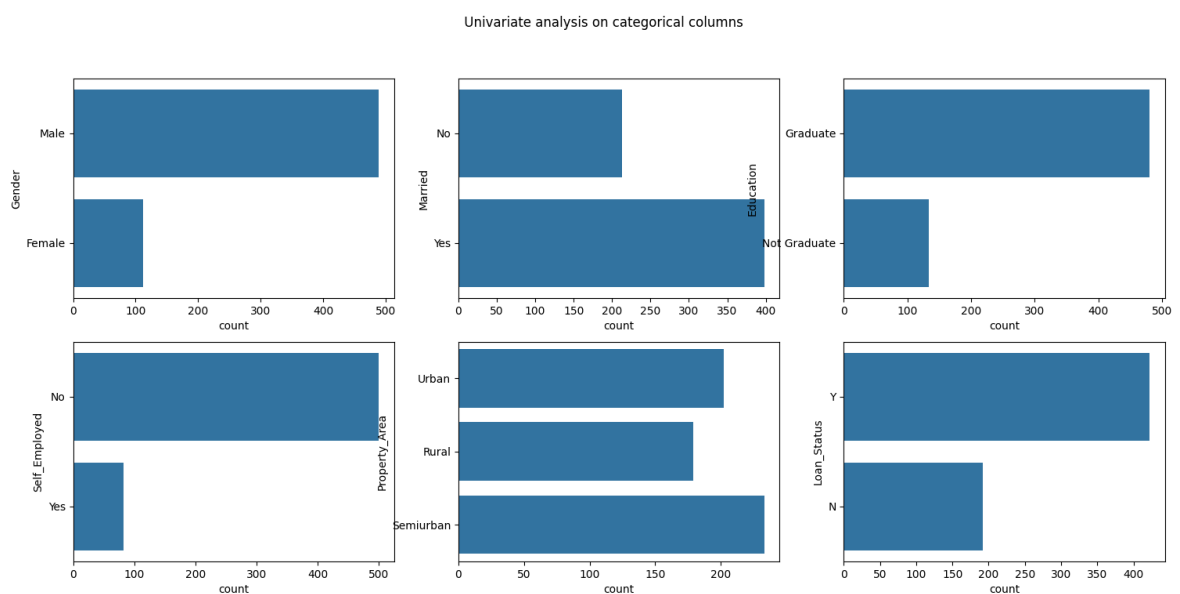
plt.subplot(2,3,4)
sns.countplot(data['Self_Employed'])

plt.subplot(2,3,5)
sns.countplot(data['Property_Area'])

plt.subplot(2,3,6)
sns.countplot(data['Loan_Status'])

plt.suptitle(' Univariate analysis on categorical columns')
plt.show()

```



For the discrete variables applied the crosstab between the 'Loan_Status' and other discrete categorical variables

```

In [54]: print("Impact of Marriage on Loan_Status")
print(pd.crosstab(data['Loan_Status'], data['Married']))
print('\n')

print("Impact of Dependents on Loan_Status")
print(pd.crosstab(data['Loan_Status'], data['Dependents']))
print('\n')

print("Impact of Education on Loan_Status")
print(pd.crosstab(data['Loan_Status'], data['Education']))
print('\n')

print("Impact of Employment on Loan_Status")
print(pd.crosstab(data['Loan_Status'], data['Self_Employed']))
print('\n')

print("Impact of Property on Loan_Status")

```



```
print(pd.crosstab(data['Loan_Status'],data['Property_Area']))
print('\n')
```

Impact of Marriage on Loan_Status

Married	No	Yes
Loan_Status		
N	79	113
Y	134	285

Impact of Dependents on Loan_Status

Dependents	0	1	2	3+
Loan_Status				
N	107	36	25	18
Y	238	66	76	33

Impact of Education on Loan_Status

Education	Graduate	Not Graduate
Loan_Status		
N	140	52
Y	340	82

Impact of Employment on Loan_Status

Self_Employed	No	Yes
Loan_Status		
N	157	26
Y	343	56

Impact of Property on Loan_Status

Property_Area	Rural	Semiurban	Urban
Loan_Status			
N	69	54	69
Y	110	179	133

Checked for Missing Values

```
In [56]: data.isnull().sum()
```

```
Out[56]: Gender          13
Married              3
Dependents          15
Education            0
Self_Employed       32
LoanAmount          22
Loan_Amount_Term     14
Credit_History      50
Property_Area        0
Loan_Status          0
Income              0
dtype: int64
```

Checked for Missing Skewness

```
In [58]: data[continuous].skew()
```

```
Out[58]: Income      5.633449
LoanAmount  2.677552
dtype: float64
```

Those skewness values indicate that both the Income and LoanAmount distributions are highly right-skewed (positively skewed)

What skewness means numerically: Skewness = 0 → perfectly symmetrical (e.g., bell-shaped)

Skewness > 1 → highly positively skewed

Skewness between 0.5 and 1 → moderately skewed

Skewness < -1 → highly negatively skewed

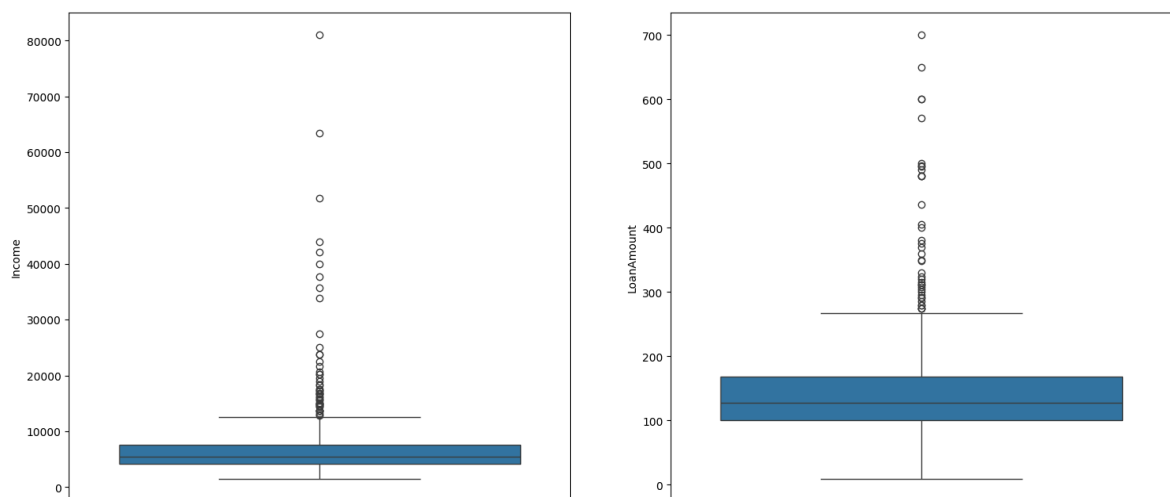
Checked for Outliers

```
In [62]: plt.subplot(1,2,1)
sns.boxplot(data['Income'])

plt.subplot(1,2,2)
sns.boxplot(data['LoanAmount'])

plt.suptitle('Outliers in the Data')
plt.show()
```

Outliers in the Data



Income & LoanAmount Both Having Outliers ---> but we cant consider them as outliers as some people can have the high income or can apply high loan amount

3) Data Preparation

1)- Data Cleaning --> 1) Wrong Data 2) Missing Values 3) Wrong data types 4) Duplicates 5) Outliers

2)- Data Wrangling --> 1) Transformation 2) Scaling 3) Encoding

1) Data Cleaning

Wrong Data Treatment

```
In [68]: data['Dependents'] = data['Dependents'].replace({'3+':3})
```

```
In [69]: data['Dependents'].unique()
```

```
Out[69]: array(['0', '1', '2', 3, nan], dtype=object)
```

Missing values Treatment

```
In [71]: data['Dependents'] = data['Dependents'].fillna(0)

data['Gender'] = data['Gender'].fillna(data['Gender'].mode()[0])

data['Married'] = data['Married'].fillna(data['Married'].mode()[0])

data['Self_Employed'] = data['Self_Employed'].fillna(data['Self_Employed'].mode(

data = data.dropna(subset = ['Income', 'LoanAmount', 'Loan_Amount_Term', 'Credit
# As Loan will be depending on these variables so we can't replace these variabl
```

Data Types Conversions

```
In [73]: data['Dependents'] = data['Dependents'].astype('int')

data['Loan_Amount_Term'] = data['Loan_Amount_Term'].astype('int')
```

Duplicates Treatments

```
In [75]: data.duplicated().sum()
```

```
Out[75]: 0
```

Outliers Treatments

```
In [77]: # Outliers should be retrained(Really some people will have high income)
```

2) Data Wrangling

Encoding

```
In [80]: data.columns
```

```
Out[80]: Index(['Gender', 'Married', 'Dependents', 'Education', 'Self_Employed',
               'LoanAmount', 'Loan_Amount_Term', 'Credit_History', 'Property_Area',
               'Loan_Status', 'Income'],
              dtype='object')
```

```
In [81]: data['Gender'] = data['Gender'].replace({'Male':1, 'Female':0})
data['Married'] = data['Married'].replace({'Yes':1, 'No':0})
data['Education'] = data['Education'].replace({'Graduate':1, 'Not Graduate':0})
data['Self_Employed'] = data['Self_Employed'].replace({'Yes':1, 'No':0})
data['Credit_History'] = data['Credit_History'].replace({'good':1, 'bad':0})
```

```
data['Property_Area'] = data['Property_Area'].replace({'Rural':0, 'Semiurban':1},
data['Loan_Status'] = data['Loan_Status'].replace({'Y':1, 'N':0})
```

Transformations

for the right skewed---> Go for Transformation

```
In [83]: from scipy.stats import boxcox
data['Income'], a = boxcox(data['Income'])
data['LoanAmount'], c = boxcox(data['LoanAmount'])
```

Checked the skewness for Income & LoanAmount

```
In [85]: data['Income'].skew()
```

```
Out[85]: -0.02776908801128399
```

```
In [86]: data['LoanAmount'].skew()
```

```
Out[86]: 0.03828926265578848
```

Scaling

I have checked Income & LoanAmount --> Both have same scale ..so no need to apply scaling on that

Also no need to do scaling on count variables

X & y

```
In [90]: X = data.drop('Loan_Status', axis=1)
y = data['Loan_Status']
```

```
In [91]: y = data['Loan_Status']
```

```
In [92]: y.value_counts()
```

```
Out[92]: Loan_Status
1      366
0      163
Name: count, dtype: int64
```

train-test split

```
In [94]: from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

4) Modeling & Evaluation

```
In [96]: from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
```

```

from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.ensemble import GradientBoostingClassifier
from xgboost import XGBClassifier

from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score
from sklearn.model_selection import cross_val_score

```

1) Logistic Regression

In [98]: *# Modeling*

```

log_model = LogisticRegression()
log_model.fit(X_train, y_train)

```

Out[98]:

LogisticRegression ⓘ ?

LogisticRegression()

In [99]: *# Evaluation*

```

ypred_train = log_model.predict(X_train)
ypred_test = log_model.predict(X_test)

```

In [100...]

```

print("Train Accuracy: ", accuracy_score(y_train, ypred_train) )
print("CV Score: ", cross_val_score(log_model, X_train, y_train, cv=5, scoring='
print("Test Accuracy: ", accuracy_score(y_test, ypred_test))

```

Train Accuracy: 0.8226950354609929
 CV Score: 0.8227731092436976
 Test Accuracy: 0.7830188679245284

2) KNN

- HPT
- Model & Evaluation

In [103...]

```

# Hyper parameter Tuning

estimator = KNeighborsClassifier()
param_grid = {'n_neighbors': list(range(1,50))}

knn_grid = GridSearchCV(estimator, param_grid, scoring='accuracy', cv=5)
knn_grid.fit(X_train, y_train)

knn_grid.best_estimator_

```

Out[103...]

KNeighborsClassifier ⓘ ?

KNeighborsClassifier(n_neighbors=12)

In [104...]

```

# Modeling
knn_model = KNeighborsClassifier(n_neighbors=12)

```

```
knn_model.fit(X_train, y_train)
```

Out[104...

```
▼ KNeighborsClassifier ⓘ ?  
KNeighborsClassifier(n_neighbors=12)
```

In [105...

```
# Evaluation  
ypred_train = knn_model.predict(X_train)  
ypred_test = knn_model.predict(X_test)
```

In [106...

```
print("Train Accuracy: ", accuracy_score(y_train, ypred_train))  
print("CV Score: ", cross_val_score(knn_model, X_train, y_train, scoring='accuracy'))  
print("Test Accuracy: ", accuracy_score(y_test, ypred_test))
```

Train Accuracy: 0.7683215130023641

CV Score: 0.7375630252100841

Test Accuracy: 0.7358490566037735

3) SVM (Support Vector Machine)

- HPT
- Modeling & Evaluation

In [109...

```
# Hyperparameter Tuning  
  
estimator = SVC()  
param_grid = {'C':[0.01,0.1, 1], 'kernel':['linear', 'rbf', 'sigmoid', 'poly']}  
  
svm_grid = GridSearchCV(estimator, param_grid, scoring = 'accuracy', cv=5)  
svm_grid.fit(X_train, y_train)  
  
svm_grid.best_estimator_
```

Out[109...

```
▼ SVC ⓘ ?  
SVC(C=0.1, kernel='linear')
```

In [110...

```
# Modeling  
svm_model = SVC(C=0.1, kernel='linear')  
svm_model.fit(X_train, y_train)
```

Out[110...

```
▼ SVC ⓘ ?  
SVC(C=0.1, kernel='linear')
```

In [111...

```
# Evaluation  
ypred_train = svm_model.predict(X_train)  
ypred_test = svm_model.predict(X_test)  
  
print("Train accuracy: ", accuracy_score(y_train, ypred_train))  
print("CV Score: ", cross_val_score(svm_model, X_train, y_train, scoring='accuracy'))  
print("Test accuracy: ", accuracy_score(y_test, ypred_test))
```

Train accuracy: 0.8226950354609929

CV Score: 0.8227731092436976

Test accuracy: 0.7830188679245284

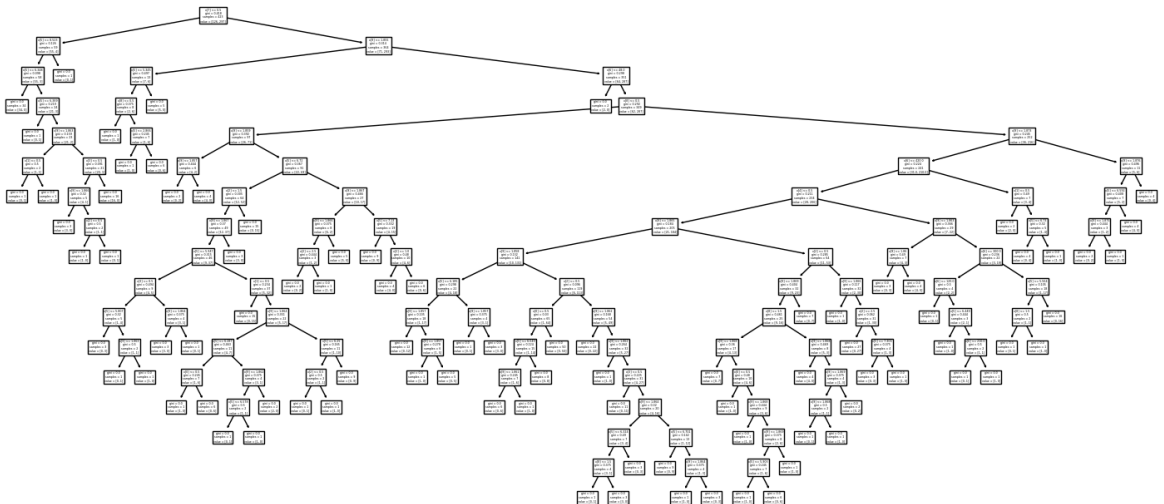
4) Decision Tree Classifier

- HPT
- Identified important feature by using feature selection (ensemble method)
- Modeling & Evaluation

```
In [114... model = DecisionTreeClassifier(random_state = True)
model.fit(X_train, y_train)
```

```
Out[114... ▼ DecisionTreeClassifier
DecisionTreeClassifier(random_state=True)
```

```
In [115... from sklearn.tree import plot_tree
plot_tree(model)
plt.show()
```



```
In [116... # Hyperparameter Tuning

estimator = DecisionTreeClassifier(random_state=True)
param_grid = {'criterion': ['gini', 'entropy'],
              'max_depth': list(range(1,16))}

dt_grid = GridSearchCV(estimator, param_grid, scoring = 'accuracy', cv=5 )
dt_grid.fit(X_train, y_train)

dt = dt_grid.best_estimator_
dt.feature_importances_
```

```
Out[116... array([0., 0., 0., 0., 0., 0., 0., 1., 0., 0.])
```

```
In [117... # Important Parameters

feats_dt = pd.DataFrame(data = dt.feature_importances_,
```

```

        index = X.columns,
        columns=['Importance'] )

important_features_dt = feats_dt[feats_dt['Importance']>0].index.tolist()

important_features_dt

```

Out[117... ['Credit_History']

Creating Decision Tree Model with important parameters and important features

```

In [119... # Selecting train & test data

X_train_dt = X_train[important_features_dt]
X_test_dt = X_test[important_features_dt]

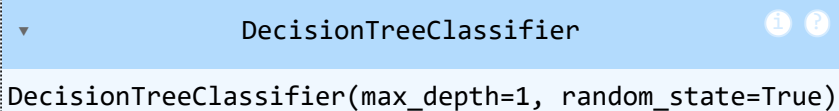
```

```

In [120... # Modeling

dt.fit(X_train_dt, y_train)

```

Out[120...  DecisionTreeClassifier(max_depth=1, random_state=True)

```

In [121... # Evaluation

ypred_train = dt.predict(X_train_dt)
ypred_test = dt.predict(X_test_dt)

```

```

In [122... print("Train accuracy: ", accuracy_score(y_train, ypred_train))
print("CV Score: ", cross_val_score(dt, X_train, y_train, scoring='accuracy', cv
print("Test accuracy: ", accuracy_score(y_test, ypred_test))

```

Train accuracy: 0.8226950354609929

CV Score: 0.8227731092436976

Test accuracy: 0.7830188679245284

5) Random Forest Classifier

- HPT
- Identified important feature by using feature selection (ensemble method)
- Modeling & Evaluation

```

In [125... # Hyperparameter Tuning

estimator = RandomForestClassifier(random_state=True)
param_grid = {'n_estimators': list(range(1, 51))}

rf_grid = GridSearchCV(estimator, param_grid, scoring='accuracy', cv=5)
rf_grid.fit(X_train, y_train)
rf = rf_grid.best_estimator_
rf

```


Out[125...

RandomForestClassifier

RandomForestClassifier(n_estimators=21, random_state=True)

In [126...

```
# Important features

feats_ab = pd.DataFrame(data = rf.feature_importances_,
                        index = X.columns,
                        columns = ['Importance'])

important_features_rf = feats_ab[feats_ab['Importance']>0].index.tolist()
important_features_rf
```

Out[126...

```
['Gender',
 'Married',
 'Dependents',
 'Education',
 'Self_Employed',
 'LoanAmount',
 'Loan_Amount_Term',
 'Credit_History',
 'Property_Area',
 'Income']
```

In [127...

```
# Selecting train & test data

X_train_rf = X_train[important_features_rf]
X_test_rf = X_test[important_features_rf]
```

In [128...

```
# Modeling

rf.fit(X_train_rf, y_train)
```

Out[128...

RandomForestClassifier

RandomForestClassifier(n_estimators=21, random_state=True)

In [129...

```
# Evaluation

ypred_train = rf.predict(X_train_rf)
ypred_test = rf.predict(X_test_rf)
```

In [130...

```
print("Train accuracy: ", accuracy_score(y_train, ypred_train))
print("CV Score: ", cross_val_score(rf, X_train, y_train, scoring='accuracy', cv
print("Test accuracy: ", accuracy_score(y_test, ypred_test))
```

Train accuracy: 0.9952718676122931

CV Score: 0.8203641456582634

Test accuracy: 0.7735849056603774

6) AdaBoost Classifier

- HPT

- Identified important feature by using feature selection (ensemble method)
- Modeling & Evaluation

```
In [133... # Hyperparameter Tuning

estimator = AdaBoostClassifier(random_state=True)
param_grid = {'n_estimators': list(range(1, 51))}

ab_grid = GridSearchCV(estimator, param_grid, scoring='accuracy', cv=5)
ab_grid.fit(X_train, y_train)
ab = ab_grid.best_estimator_
ab
```

```
Out[133... ▼ AdaBoostClassifier ⓘ ?
AdaBoostClassifier(n_estimators=3, random_state=True)
```

```
In [134... # Important features

feats_ab = pd.DataFrame(data = ab.feature_importances_,
                        index = X.columns,
                        columns = ['Importance'])

important_features_ab = feats_ab[feats_ab['Importance']>0].index.tolist()
important_features_ab
```

```
Out[134... ['Credit_History', 'Property_Area', 'Income']
```

```
In [135... # Selecting train & test data

X_train_ab = X_train[important_features_ab]
X_test_ab = X_test[important_features_ab]
```

```
In [136... # Modeling

ab.fit(X_train_ab, y_train)
```

```
Out[136... ▼ AdaBoostClassifier ⓘ ?
AdaBoostClassifier(n_estimators=3, random_state=True)
```

```
In [137... # Evaluation

ypred_train = ab.predict(X_train_ab)
ypred_test = ab.predict(X_test_ab)
```

```
In [138... print("Train accuracy: ", accuracy_score(y_train, ypred_train))
print("CV Score: ", cross_val_score(ab, X_train, y_train, scoring='accuracy', cv
print("Test accuracy: ", accuracy_score(y_test, ypred_test))
```

```
Train accuracy: 0.83451536643026
CV Score: 0.8322408963585435
Test accuracy: 0.7641509433962265
```

7) Gradient Boost Classifier

- HPT
- Identified important feature by using feature selection (ensemble method)
- Modeling & Evaluation

```
In [141... # Hyperparameter Tuning

estimator = GradientBoostingClassifier(random_state=True)
param_grid = {'n_estimators': list(range(1, 10)),
              'learning_rate': [0.1, 0.2, 0.3, 0.4, 0.5, 0.7, 0.8, 0.9, 1.0]}

gb_grid = GridSearchCV(estimator, param_grid, scoring='accuracy', cv=5)
gb_grid.fit(X_train, y_train)
gb = gb_grid.best_estimator_
gb
```

```
Out[141... GradientBoostingClassifier

GradientBoostingClassifier(learning_rate=0.3, n_estimators=3, random_state=True)
```

```
In [142... # Important features

feats_gb = pd.DataFrame(data = gb.feature_importances_,
                        index = X.columns,
                        columns = ['Importance'])

important_features_gb = feats_gb[feats_gb['Importance']>0].index.tolist()
important_features_gb
```

```
Out[142... ['Married',
             'LoanAmount',
             'Loan_Amount_Term',
             'Credit_History',
             'Property_Area',
             'Income']
```

```
In [143... # Selecting train & test data

X_train_gb = X_train[important_features_gb]
X_test_gb = X_test[important_features_gb]
```

```
In [144... # Modeling

gb.fit(X_train_gb, y_train)
```

```
Out[144... GradientBoostingClassifier

GradientBoostingClassifier(learning_rate=0.3, n_estimators=3, random_state=True)
```

In [145...

Evaluation

```
ypred_train = gb.predict(X_train_gb)
ypred_test = gb.predict(X_test_gb)
```

In [146...

```
print("Train accuracy: ", accuracy_score(y_train, ypred_train))
print("CV Score: ", cross_val_score(gb, X_train, y_train, scoring='accuracy', cv=5))
print("Test accuracy: ", accuracy_score(y_test, ypred_test))
```

Train accuracy: 0.8321513002364066

CV Score: 0.8227450980392158

Test accuracy: 0.7735849056603774

8) XG Boost Classifier

- HPT
- Identified important feature by using feature selection (ensemble method)
- Modeling & Evaluation

In [149...

Hyperparameter Tuning

```
estimator = XGBClassifier()
param_grid = {'n_estimators': [10, 20, 40, 100],
              'max_depth': [3, 4, 5],
              'gamma': [0, 0.15, 0.3, 0.5, 1]}

xgb_grid = GridSearchCV(estimator, param_grid, scoring='accuracy', cv=5)
xgb_grid.fit(X_train, y_train)
xgb = xgb_grid.best_estimator_
xgb
```

Out[149...

XGBClassifier

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              feature_weights=None, gamma=0.3, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=None, max_bin=None, max_cat_threshold=None,
              max_delta_step=None, max_leaf_nodes=None, min_child_weight=None,
              missing=None, multi_output=False, n_estimators=None, num_parallel_tree=None,
              random_state=None, reg_alpha=None, reg_lambda=None, scale_pos_weight=None,
              subsample=None, tree_method=None, validate_parameters=None, verbosity=None,
              watchlist=None, weight_posterior=None)
```

In [150...

Important features

```
feats_xgb = pd.DataFrame(data = xgb.feature_importances_,
                        index = X.columns,
                        columns = ['Importance'])

important_features_xgb = feats_xgb[feats_xgb['Importance'] > 0].index.tolist()
important_features_xgb
```

```
Out[150...] ['Dependents',
             'Self_Employed',
             'LoanAmount',
             'Loan_Amount_Term',
             'Credit_History',
             'Property_Area',
             'Income']
```

```
In [151...] # Selecting train & test data
```

```
X_train_xgb = X_train[important_features_xgb]
X_test_xgb = X_test[important_features_xgb]
```

```
In [152...] # Modeling
```

```
xgb.fit(X_train_xgb, y_train)
```

```
Out[152...] XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              feature_weights=None, gamma=0.3, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=None, max_bin=None, max_cat_threshold=None,
              max_delta_step=None, max_depth=None, min_child_weight=None,
              missing=None, multi_class_strategy=None, num_parallel_tree=None,
              n_estimators=None, n_jobs=None, num_threads=None,
              print_eval_method=None, random_state=None,
              reg_alpha=None, reg_lambda=None,
              scale_pos_weight=None, subsample=None,
              tree_method=None,
              validate_each_iter=False,
              validate_each_iter_timesteps=None,
              validate_log=None,
              validate_metric=None,
              validate_name=None,
              validate_score_function=None,
              verbose=None,
              watchlog=None,
              watchmetric=None)
```

```
In [153...] # Evaluation
```

```
ypred_train = xgb.predict(X_train_xgb)
ypred_test = xgb.predict(X_test_xgb)
```

```
In [154...] print("Train accuracy: ", accuracy_score(y_train, ypred_train))
print("CV Score: ", cross_val_score(xgb, X_train, y_train, scoring='accuracy', cv=5))
print("Test accuracy: ", accuracy_score(y_test, ypred_test))
```

Train accuracy: 0.8321513002364066

CV Score: 0.8251260504201682

Test accuracy: 0.7830188679245284

Save the best model

```
In [156...] from joblib import dump
dump(dt, 'loan.joblib')
```

```
Out[156...] ['loan.joblib']
```

Predict on new data

```
In [158... input_data = {
    "Loan_ID": "LP002991",
    "Gender": "Male",
    "Married": "No",
    "Dependents": 1,
    "Education": "Graduate",
    "Self_Employed": "yes",
    "ApplicantIncome": 1000,
    "CoapplicantIncome": 0,
    "LoanAmount": 100,
    "Loan_Amount_Term": 240,
    "Credit_History": "bad",
    "Property_Area": "Urban"
}
```

```
In [159... data = pd.DataFrame(input_data, index=[0])
data
```

```
Out[159...      Loan_ID  Gender  Married  Dependents  Education  Self_Employed  ApplicantIncome
0  LP002991   Male      No           1   Graduate             yes             1000
```

Apply Data Preprocessing on Unknown Data

```
In [161... # dropped unimportant column
data.drop(columns=["Loan_ID"], inplace=True)

# Created new column
data["Income"] = data["ApplicantIncome"] + data["CoapplicantIncome"]
data.drop(columns=['ApplicantIncome', 'CoapplicantIncome'], inplace=True)

# Handeled missing values
data['Dependents'] = data['Dependents'].fillna(0)
data['Gender'] = data['Gender'].fillna(data['Gender'].mode()[0])
data['Married'] = data['Married'].fillna(data['Married'].mode()[0])
data['Self_Employed'] = data['Self_Employed'].fillna(data['Self_Employed'].mode()[0])
data = data.dropna(subset=["Income", 'LoanAmount', 'Loan_Amount_Term', 'Credit_Hist

# Treated wrong data types
data['Dependents'] = data['Dependents'].astype('int')
data['Loan_Amount_Term'] = data['Loan_Amount_Term'].astype('int')

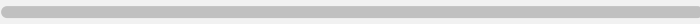
# Encoding
data['Gender'] = data['Gender'].replace({'Male':1, 'Female':0})
data['Married'] = data['Married'].replace({'Yes':1, 'No':0})
data['Education'] = data['Education'].replace({'Graduate':1, 'Not Graduate':0})
data['Self_Employed'] = data['Self_Employed'].replace({'Yes':1, 'No':0})
data['Property_Area'] = data['Property_Area'].replace({'Rural':0, 'Semiurban':1, 'Urban':2})
data['Credit_History'] = data['Credit_History'].replace({'good':1, 'bad':0})

X_new = data
```

```
In [162... X_new
```

Out[162...

	Gender	Married	Dependents	Education	Self_Employed	LoanAmount	Loan_Amour
0	1	0	1	1	yes	100	

<  >

Select important features of your best model

In [164... `X_new = X_new[['Credit_History']]`

Apply and predict using your best model

In [166... `dt.predict(X_new)`

Out[166... `array([0], dtype=int64)`

i.e Not Eligible